

Detección de velocidad transaccional

Kafka y Apache Beam · DirectRunner
Fátima Barrios Ortega

La regla detecta concentración explicable

Problema

Movimientos fragmentados pueden ocultar una secuencia relevante.

Usuario

Analistas de monitoreo priorizan clientes y revisan contexto.

Conteo

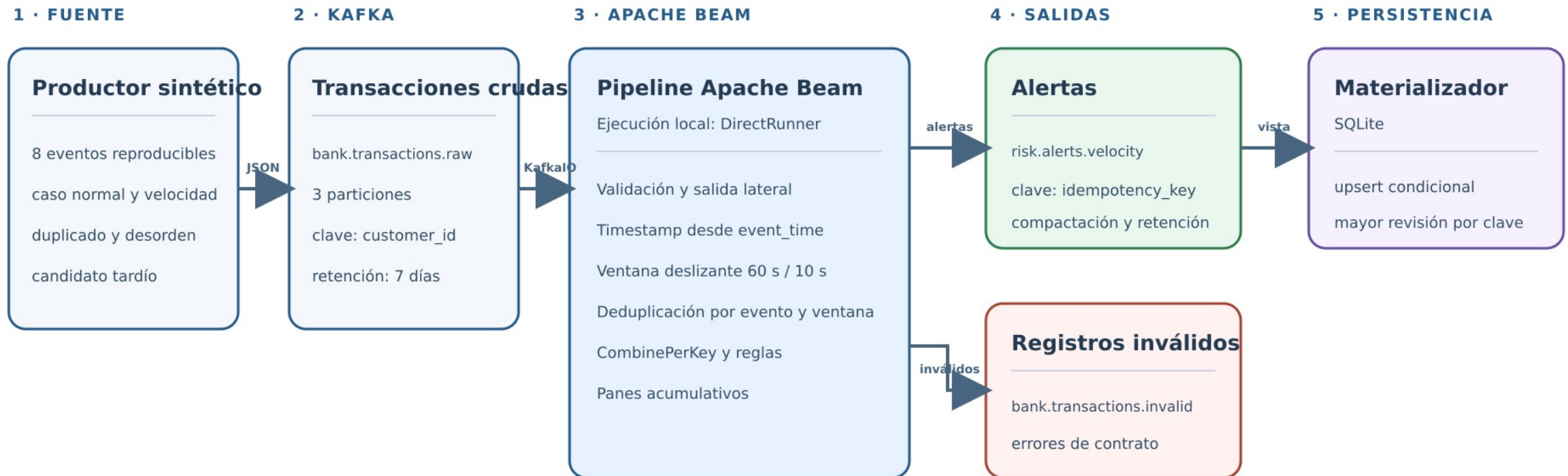
Alerta con 4 o más transacciones dentro de la ventana.

Monto

Alerta con 10.000.000 PYG o más acumulados.

El flujo separa transporte, cómputo y salida

Una clave de cliente mantiene localidad y orden; Beam aplica la semántica temporal y escribe efectos estables.



Garantía declarada: entrega at-least-once con deduplicación e idempotencia.
No se afirma exactly-once de extremo a extremo entre Kafka y SQLite.

Claves y contrato sostienen el orden

customer_id conserva localidad en el input; idempotency_key permite converger en la salida.

Tópico	Part.	Clave	Política	Propósito
bank.transactions.raw	3	customer_id	7 días	entrada y replay
risk.alerts.velocity	1	idempotency_key	compact, delete	revisiones
bank.transactions.invalid	1	event_id/hash	7 días	errores

Beam mantiene cada decisión en un módulo

1. Leer

KafkaIO y grupo estable

2. Validar

JSON, UTF-8 y contrato

3. Timestamp

event_time del payload

4. Ventana

deslizante 60 s / 10 s

5. Deduplicar

event_id por ventana

6. Agregar

CombinePerKey

7. Alertar

conteo o monto

8. Escribir

alertas e inválidos

La política temporal corrige sin estado infinito

Tiempo de evento

El timestamp viene de la transacción, no de la llegada.

Ventana

60 segundos con avance cada 10 segundos.

Tardanza permitida

30 segundos para aceptar correcciones acotadas.

Panes

EARLY, ON_TIME y LATE en modo ACCUMULATING.

La confiabilidad converge por clave estable

Deduplicar

event_id por cliente/ventana

Temporizador

limpia en fin + 30 s

Clave

cliente + inicio + fin

Revisión

pane_index creciente

Acumular

total vigente, no delta

Compactar

misma clave en Kafka

Materializar

mayor revisión

Semántica

at-least-once idempotente

TestStream valida duplicados, tardíos y cierre

01	Watermark llega al inicio de la ventana
02	Tres eventos únicos y un duplicado
03	Watermark alcanza window_end: pane ON_TIME
04	Evento tardío aceptado: pane LATE acumulado
05	Watermark supera allowed lateness de 30 s
06	Evento demasiado tardío se descarta y cierra estado

La validación confirmó el recorrido completo

DirectRunner finalizó con código 0; Kafka confirmó consumo, escritura y ausencia de inválidos.



71

pruebas automatizadas
Ruff sin hallazgos



8

eventos consumidos
LAG final = 0



3 / 0

alertas / inválidos
PIPELINE_EXIT_CODE=0

Tres ventanas activaron la regla esperada

La clave Kafka coincide con alert_id e idempotency_key; los tres mensajes son ON_TIME y ACCUMULATING.

Offset	Ventana UTC	Conteo	Total PYG	Condición
0	19:59:50-20:00:50	5	10.500.000	conteo + monto
1	19:59:40-20:00:40	4	8.000.000	conteo
2	20:00:00-20:01:00	4	8.000.000	conteo

Los límites quedan explícitos

Entorno local

DirectRunner sirve para desarrollo y prueba, no para producción distribuida.

Watermark

La ejecución acotada cierra ON_TIME; TestStream valida los tardíos.

Infraestructura

Un broker, RF=1 y una partición de salida.

Siguiente paso

Runner distribuido, checkpoints, métricas y umbrales por segmento.

CONCLUSIÓN

**Resultado validado.
Alcance documentado.**

Kafka → Beam → Kafka

8 eventos · 3 alertas · 0 inválidos

Fátima Barrios Ortega